our shielding · Your smart contracts, our shielding · Your smart c

shieldify

PrimeV

Fast Settlement V3

SECURITY REVIEW

Date: 7 February 2026

CONTENTS

1. About Shieldify **3**

2. Disclaimer **3**

3. About PrimeV – Fast Settlement V3 **3**

4. Risk classification **4**

4.1 Impact 4

4.2 Likelihood 4

5. Security Review Summary **4**

5.1 Protocol Summary 5

5.2 Scope 5

6. Findings Summary **5**

7. Findings **6**

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About PrimeV - Fast Settlement V3

Overview

FastSettlementV3 is a UUPS upgradeable intent settlement contract that executes token swaps on behalf of users with guaranteed minimum output amounts.

Key Actors

Actor	Description
User	Signs an intent expressing swap parameters and minimum output
Executor	Whitelisted address that submits intents + swap calldata
Treasury	Receives surplus above user's minimum output

Entry Points

Path 1: `executeWithPermit()` **(Executor submits on behalf of the user)**

```
User signs Intent + Permit2 signature → FastSwap API↓

API fetches optimal swap route from DEX aggregator↓

Executor wallet calls executeWithPermit(intent, signature, swapData)↓

Permit2 pulls ERC20 from user → swap → Verify output >= userAmtOut↓

Send `userAmtOut` to the recipient, surplus to the treasury
```

Path 2: `executeWithETH()` **(User submits directly)**

```
User requests swap data → FastSwap API returns unsigned tx↓

User signs and submits `executeWithETH{value: inputAmt}(intent, swapData)
    `↓

Wrap ETH → WETH → Execute swap → Verify output >= userAmtOut↓

Send `userAmtOut` to the recipient, surplus to the treasury
```

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 3 days, with a total of 48 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying one Medium and one Low severity issues. They're mainly related to execution-path edge cases and asset handling inconsistencies affecting output guarantees and refunds

The PrimeV team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	**PrimeV - Fast Settlement V3**
Repository	fastprotocolapp
Type of Project	DeFi, Infrastructure
Security Review Timeline	3 days
Review Commit Hash	dc5b0e0b5df3b26e2c6fbdaf75d382f4f58b6066

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
contracts/src/FastSettlementV3.sol	173
Total	173

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: **1**
- **Low** issues: **1**
- **Info** issues: **1**

ID	Title	Severity	Status
[M-01]	Fee-on-Transfer Tokens Break User Output Guarantees and Accounting	Medium	Acknowledged
[L-01]	Recipient Will Lose Output Token if Price Changes in Favour of the Recipient	Low	Acknowledged
[I-01]	ETH Refund Not Unwrapped in `executeWithETH()`	Info	Acknowledged

[M-01] Fee-on-Transfer Tokens Break User Output Guarantees and Accounting

Severity

Medium Risk

Description

The `FastSettlementV3` contract does not account for fee-on-transfer (FOT) tokens, which deduct a percentage on every `transfer()` or `transferFrom()` call. When FOT tokens are used as input or

output tokens, the actual amounts received differ from expected amounts, causing users to receive less than their signed `userAmtOut` and breaking the core promise of the intent system.

Fee-on-transfer tokens (also called deflationary or tax tokens) are prevalent on the Ethereum mainnet. Common examples include:

- **SafeMoon** (and numerous forks) – 10% fee
- **PAXG** (Paxos Gold) – 0.02% fee
- **USDT** – Has a fee mechanism built in (currently set to 0, but can be enabled)
- Various meme/reflection tokens – 1-10% fees

The protocol assumes that when it calls `safeTransfer(recipient, amount)`, the recipient receives exactly `amount` tokens. This assumption breaks for FOT tokens, where a percentage is deducted during the transfer, resulting in users receiving less than the `userAmtOut` they cryptographically signed for.

Industry Reference: 1inch Protocol Handling

The 1inch protocol, a major DEX aggregator, explicitly acknowledges FOT token issues in its official documentation. According to the 1inch API Troubleshooting Guide, under the "Cannot Estimate" error section:

> **Some common reasons why a transaction may fail:**
>
> - A token has a fee on transfer or swap and the slippage tolerance needs to be increased
> - A token has a fee on transfer and the fee and referrer parameter is set, causing the transaction to always fail

The 1inch `SwapFacade` contract (0x179dc3fb0f2230094894317f307241a52cdb38aa) uses a `MinReturnInfo` struct that validates the **recipient's actual balance** after swaps:

```
// From 1inch SwapFacade.sol
struct MinReturnInfo {
    IERC20 token;
    uint256 amount;
    address recipient;
}

function _swap(...) private returns (uint256[] memory) {
   uint256[] memory balancesBeforeSwap = new uint256[](minReturns.length)
      ;
   for (uint256 i = 0; i < minReturns.length; i++) {
      balancesBeforeSwap[i] = minReturns[i].token.universalBalanceOf(
         minReturns[i].recipient);
   }

   executor.executeSwap{value: msg.value}(targetTokenTransferInfos,
      swapDescriptions);

   for (uint256 i = 0; i < minReturns.length; i++) {
      uint256 balanceAfterSwap = minReturns[i].token.universalBalanceOf(
         minReturns[i].recipient);
      uint256 totalSwappedAmount = balanceAfterSwap - balancesBeforeSwap[
         i];
      if (totalSwappedAmount < minReturns[i].amount) {
         revert MinReturnError(totalSwappedAmount, minReturns[i].amount);
      }
   }
}
```

Location of Affected Code

File: contracts/src/FastSettlementV3.sol

Output token transfer to recipient

```
function _execute( Intent calldata intent, SwapCall calldata swapData,
   address actualInputToken ) internal returns (uint256 received, uint256
    surplus) {
   // code
   // Pay user
   if (outputToken == address(0)) {
      payable(intent.recipient).sendValue(intent.userAmtOut);
   } else {
      IERC20(outputToken).safeTransfer(intent.recipient, intent.
         userAmtOut);
   }
   // code
}
```

Surplus transfer to treasury

```
function _execute( Intent calldata intent, SwapCall calldata swapData,
   address actualInputToken ) internal returns (uint256 received, uint256
    surplus) {
   // code
   // Send surplus to treasury
   if (surplus > 0) {
      if (outputToken == address(0)) {
         payable(treasury).sendValue(surplus);
      } else {
         IERC20(outputToken).safeTransfer(treasury, surplus);
      }
      // code
   }
   // code
}
```

Input token pull via Permit2

```
function _pullWithPermit2(Intent calldata intent, bytes calldata
   signature) internal {
    IPermit2.PermitTransferFrom memory permit = IPermit2.
       PermitTransferFrom({
        permitted: IPermit2.TokenPermissions({
            token: intent.inputToken,
            amount: intent.inputAmt
        }),
        nonce: intent.nonce,
        deadline: intent.deadline
    });

    IPermit2.SignatureTransferDetails memory transferDetails = IPermit2
        .SignatureTransferDetails({to: address(this), requestedAmount:
           intent.inputAmt});
    // Contract requests intent.inputAmt but receives less due to FOT fee
    // code
}
```

Input token pull via Permit2

```
function _execute( Intent calldata intent, SwapCall calldata swapData,
   address actualInputToken ) internal returns (uint256 received, uint256
    surplus) {
   // code
   // Refund unused input
   uint256 finalInputBal = _getBalance(actualInputToken);
   if (finalInputBal > startInputBal) {
      uint256 unused = finalInputBal - startInputBal;
      IERC20(actualInputToken).safeTransfer(intent.user, unused);
   }
   // code
}
```

Impact

Fee-on-transfer tokens break the core intent promise: users receive less than their signed `userAmtOut` with silent failures and no revert. Event logs and accounting records become unreliable, creating discrepancies for both user refunds and treasury surplus. Input token FOT fees can cause swap failures or insufficient input amounts, while users consistently receive less than expected across all settlement paths.

Recommendation

Option 1: Document and Accept (Minimal Effort)

Add explicit documentation that FOT tokens are not supported:

```
/// @title FastSettlementV3
/// @notice V3 implementation using UUPS Upgradeable pattern with dual
    entry points.
/// @dev WARNING: Fee-on-transfer (FOT) tokens are NOT SUPPORTED. Using
    FOT tokens
/// as input or output may result in users receiving less than their
    signed
/// userAmtOut amount. The protocol makes no guarantees for FOT token
    swaps.
contract FastSettlementV3 is ...
```

Option 2: Verify Actual Received Amounts (Recommended)

Follow the 1inch pattern of checking recipient balances after transfers:

```
    // Pay user with verification
    if (outputToken == address(0)) {
        payable(intent.recipient).sendValue(intent.userAmtOut);
    } else {
        uint256 recipientBalBefore = IERC20(outputToken).balanceOf(intent.
            recipient);
        IERC20(outputToken).safeTransfer(intent.recipient, intent.
            userAmtOut);
        uint256 actualReceived = IERC20(outputToken).balanceOf(intent.
            recipient) - recipientBalBefore;
        if (actualReceived < intent.userAmtOut) {
            revert InsufficientTransferToUser(actualReceived, intent.
                userAmtOut);
        }
    }
}
```

Team Response

Acknowledged.

[L-01] Recipient Will Lose Output Token if Price Changes in Favour of the Recipient

Severity

Low Risk

Description

Protocol members can maliciously manipulate users' assets and transfer them to tressury due to the 'surplus' feature. The reason is the difference between how the `outputToken` is interpreted and how it is used in the `FastSettlementV3` contract. The **FastSettlementV3 Contract Overview** document says that Intent.userAmtOut is: minimum output guaranteed to the user.

Here can be such a case where a sudden tx leads the price of inputToken higher so that the user should receive much more than `userAmtOut`. But the surplus logic does not let the user get that amount, it gives that protocol. The minimum output guaranteed means the output amount the user will definitely get, it does not mean that this tis he exact amount which the user wants to receive. The contract acts in such a way that it uses `userAmtOut` as a fixed, hardcoded amount, user can't get more/less than it.

The protocol members can exploit the situation like this: Assume a scenario: Bob is using the UniswapV2 pool. The pair is tokenA:tokenB. In that pool amount of tokenA is 1200 and the amount of tokenB is 300. So, 1 tokenB = 4 tokenA.

1. Bob wants to swap 20 tokenA, he is expected to receive 5 tokenB. So what he does is set the `userAmtOut` to 5.
2. Now, suddenly, before Bob sends his transaction, Alice executed her swap for 1000 tokenA. It is a huge amount and for this Alice received 250 tokenB.
3. Now the pool has 200 tokenA and 550 tokenB. In this stage, 1 tokenB = ~0.03 tokenA.
4. Bob's tx executed now, as he is swapping 20 tokenA, he is about to get ~666 tokenB.
5. Remember Bob sets his `userAmtOut` to 5 tokenB.
6. For that reason, Bob will only receive 5 tokenB and (666 - 5) = 661 tokenB will be transferred to the treasury.

What happened right now is totally wrong, the 666 amount of tokenB should have transferred to Bob, but he only received 5 tokenB.

Now, Alice can do it intentionally (if she is a protocol member) or unintentionally.

Location of Affected Code

File: contracts/src/FastSettlementV3.sol#L124-L180

```
function _execute( Intent calldata intent, SwapCall calldata swapData,
    address actualInputToken ) internal returns (uint256 received, uint256
     surplus) {
      // code
      surplus = received - intent.userAmtOut;

      // Pay user
      if (outputToken == address(0)) {
         payable(intent.recipient).sendValue(intent.userAmtOut);
      } else {
        IERC20(outputToken).safeTransfer(intent.recipient, intent.
           userAmtOut);
     }
     // Send surplus to treasury
     if (surplus > 0) {
        if (outputToken == address(0)) {
           payable(treasury).sendValue(surplus);
        } else {
           IERC20(outputToken).safeTransfer(treasury, surplus);
        }
      }
      // code
}
```

Impact

The recipient will lost token.

Recommendation

Use a parameter in Intent named `userAmtOutMax` where the user can set a maximum expected output amount. If the received amount is in the `userAmtOut` and `userAmtOutMax` range, then transfer the full amount to the recipient. If the received amount is more than `userAmtOutMax`, then surplus will be (`received - userAmtOutMax`). In that case, transfer the `userAmtOutMax` to the recipient and transfer the surplus to the treasury.

Team Response

Acknowledged.

[I-01] ETH Refund Not Unwrapped in `executeWithETH()`

Severity

Informational Risk

Description

In `executeWithETH()`, the user's ETH is wrapped to WETH and used for the swap. If the swap doesn't consume all the WETH (partial fill), the unused WETH remains as WETH tokens when refunded to

the user instead of being unwrapped back to native ETH. This creates an inconsistency where users send ETH but receive WETH refunds.

Location of Affected Code

File: contracts/src/FastSettlementV3.sol#L106-L120

```
function executeWithETH(Intent calldata intent, SwapCall calldata
    swapData) external payable nonReentrant returns (uint256 received,
    uint256 surplus) {
     if (intent.inputToken != address(0)) revert ExpectedETHInput();
     if (intent.user != msg.sender) revert UnauthorizedCaller();
     if (msg.value != intent.inputAmt) revert InvalidETHAmount();
     if (!allowedSwapTargets[swapData.to]) revert UnauthorizedSwapTarget()
        ;
     _validateIntent(intent);
     // Wrap ETH
     WETH.deposit{value: msg.value}();  // ETH becomes WETH
     // Execute swap using WETH
     return _execute(intent, swapData, address(WETH));
}
```

In `_execute()`, the refund logic handles the `actualInputToken` (WETH):

File: contracts/src/FastSettlementV3.sol#L165-L169

```
function _execute( Intent calldata intent, SwapCall calldata swapData,
    address actualInputToken ) internal returns (uint256 received, uint256
     surplus) {
    // code
    // Refund unused input
    uint256 finalInputBal = _getBalance(actualInputToken);
    if (finalInputBal > startInputBal) {
       uint256 unused = finalInputBal - startInputBal;
       IERC20(actualInputToken).safeTransfer(intent.user, unused);
    }
    // code
}
```

Impact

Users expect ETH refunds when sending ETH, but receive WETH tokens instead. This creates an inconsistency in denomination and forces users to incur additional gas costs to manually unwrap WETH back to ETH.

Proof of Concept

1. User calls `executeWithETH()` with 1 ETH (intent.inputAmt = 1 ETH)
2. The swap router only consumes 0.9 ETH worth of WETH
3. 0.1 WETH remains unused in the contract
4. The refund logic transfers 0.1 WETH tokens to the user
5. User now holds 0.1 WETH instead of receiving 0.1 native ETH back
6. User must perform an additional transaction to unwrap WETH ETH

Recommendation

Unwrap unused WETH back to ETH before refunding to maintain consistency with the input denomination:

```
function _execute(Intent calldata intent, SwapCall calldata swapData,
    address actualInputToken) internal returns (uint256 received, uint256
    surplus) {
     // code

     // Refund unused input
     uint256 finalInputBal = _getBalance(actualInputToken);
     if (finalInputBal > startInputBal) {
         uint256 unused = finalInputBal - startInputBal;

         // If input was originally ETH (via WETH), unwrap before refund
         if (intent.inputToken == address(0) && actualInputToken ==
            address(WETH)) {
             WETH.withdraw(unused);
             payable(intent.user).sendValue(unused);
         } else {
             IERC20(actualInputToken).safeTransfer(intent.user, unused);
         }
     }

     emit IntentExecuted(intent.user, intent.inputToken, outputToken,
        intent.inputAmt, intent.userAmtOut, received, surplus);
}
```

Team Response

Acknowledged.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Thank you!